# CECS 327 | Milestone 2

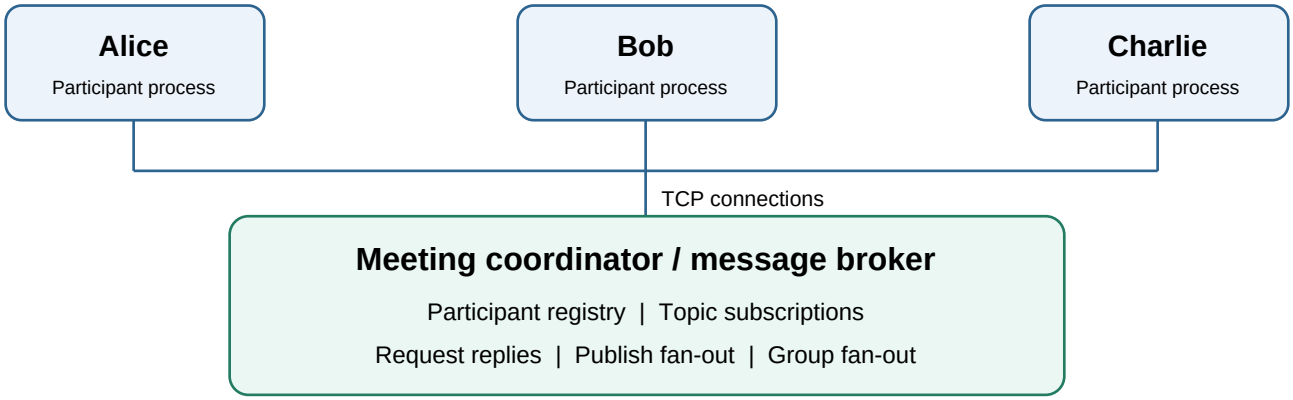
Communication mechanisms for a distributed video-meeting system

Implementation report | Python 3.9+ | TCP sockets | UTF-8 JSON | October 2026

## 1. Purpose and milestone scope

This milestone implements the communication layer of a planned Zoom-style meeting system: participant connections, request-reply exchanges, topic notifications, and meeting-wide messaging. It does not claim to stream camera/microphone data, replicate server state, elect a leader, or perform failover. Those capabilities appear in the overall project brief but are not required in Milestone 2.

## 2. Communication architecture




The server is a persistent TCP listener (server.py). Three client processes (client.py / demo_node.py) maintain independent TCP connections. The server creates a thread for each connection. In-memory registries map (meeting, username) to sessions and (meeting, topic) to subscribers.

## 3. Components

| Source module | Responsibility |
|---|---|
| protocol.py | Construct messages; marshal/unmarshal JSON; newline-delimited framing; UTC logs. |
| server.py | Accept sockets; manage membership; handle service requests; relay events and group messages. |
| client.py | Correlate replies by request ID; receive messages asynchronously; provide terminal commands. |
| demo_node.py / demo.py | Perform a deterministic four-process demonstration and save genuine logs. |
| tests/ | Unit tests and socket integration tests. |

# Protocol and communication semantics

## 4. Structured wire format

The sender constructs a Python dictionary with a message type, unique identifier, UTC creation timestamp, and body. marshal() uses json.dumps(...).encode("utf-8"), appending one newline to delimit TCP records. unmarshal() decodes the received bytes and parses one JSON object. A 65,536-byte maximum wire-message limit protects the receiver from unbounded message input.

```
Request (human-readable JSON; illustrative IDs/timestamps):
{"type":"request","id":"abc123",
 "timestamp":"2026-10-13T20:00:00.000+00:00",
 "body":{"operation":"list_participants"}}

Reply: {"type":"reply","request_id":"abc123",
        "body":{"status":"ok","participants":[...]}, ...}
```

The actual wire representation is the UTF-8 byte sequence for the compact JSON object plus the line-feed byte \n. Debug logs show both MARSHAL object=... -> wire_bytes=... and UNMARSHAL wire_bytes=... -> object=....

## 5. Request-reply model

Alice transmits a request packet with body {"operation":"list_participants"}. The broker validates the request, reads the meeting membership registry, and returns a reply packet whose request_id matches the request packet’s unique id. Each client maintains a pending-request map, allowing replies to be paired with the correct request even while event notifications arrive asynchronously.

```
Alice -> broker : request(list_participants), id=R
Broker -> Alice : reply(participants=[alice,bob,charlie]), request_id=R
```

## 6. Publish-subscribe model

Bob and Charlie issue subscribe packets naming meeting.announcements. The broker stores each registration keyed by meeting and topic. After both subscriptions are acknowledged, Alice sends publish. The broker converts it into an event and forwards it only to currently connected matching subscribers. The publisher receives a reply containing the number of socket-write deliveries.

```
Bob     -> broker : subscribe(meeting.announcements)
Charlie -> broker : subscribe(meeting.announcements)
Alice   -> broker : publish(meeting.announcements, message)
Broker  -> Bob, Charlie : event(message)
```

## 7. Group communication model

Alice sends a group_send packet with chat content. The broker looks up all currently joined members of Alice’s meeting and forwards a group_message to each, including Alice. Unlike publish-subscribe, no topic registration is necessary. Membership and topic boundaries are both scoped by meeting ID.

```
Alice -> broker : group_send("Welcome to our meeting!")
Broker -> Alice, Bob, Charlie : group_message(...)
```

# Verification and evidence

## 8. Reproducible experiments

The automated demo.py launches one server OS process and three separate client OS processes. Bob and Charlie join and subscribe before Alice publishes, preventing a startup race. The script requires success markers from every client and writes independent timestamped logs.

```
Run from project root:
  python demo.py
  python -m unittest discover -s tests -v

Observed demo result:
  PASS: request/reply, two subscriber deliveries, and group deliveries
```

## 9. Selected actual console evidence

```
[bob]
PUBSUB_RECEIVED: alice {'topic': 'meeting.announcements', 'content': 'The demo meeting has
  started.'}
GROUP_RECEIVED: alice {'content': 'Welcome to our meeting!'}
RESULT: PASS
[charlie]
PUBSUB_RECEIVED: alice {'topic': 'meeting.announcements', 'content': 'The demo meeting has
  started.'}
GROUP_RECEIVED: alice {'content': 'Welcome to our meeting!'}
RESULT: PASS
[alice]
REQUEST_REPLY: {'status': 'ok', 'operation': 'list_participants', 'meeting': 'demo-room',
  'participants': ['alice', 'bob', 'charlie']}
PUBLISH_REPLY: {'status': 'published', 'subscriber_deliveries': 2}
GROUP_SEND_REPLY: {'status': 'distributed', 'group_deliveries': 3}
RESULT: PASS
```

Unedited full machine-generated log output: evidence/demo_run/demo_transcript.txt plus each participant's *.log and *.console.txt. The logs contain actual packet IDs, timestamps, serialized bytes, and the corresponding decoded Python objects.

## 10. Test results

Automated test suite: Ran 7 tests in 0.506s; OK. Tests verify Unicode JSON round-trip, newline framing, rejection of malformed/oversize packets, request-reply, two subscriber deliveries, three group deliveries, per-meeting isolation, error replies, and duplicate-name rejection.

## 11. Limitations and subsequent work

The broker retains state only in memory and is a single point of failure. TCP ensures ordered bytes while connected; successful socket writes do not prove application-level message processing. There is no persistent event replay, participant authentication, transport encryption, server replication, election algorithm, crash recovery, or live audio/video streaming. These must not be represented as implemented Milestone 2 features.

## 12. Submission checklist

Included: runnable source, tests, README, PDF report, real logs, and a contribution template. Still pending: the recorded demonstration, the group repository link, and verified team-member contributions.